ECOLE NATIONALE DE LA STATISTIQUE ET DE L'ANALYSE DE L'INFORMATION

PROJET TRAITEMENT DE DONNEES
Etudiants 1A

Rapport du projet de traitement de données

Groupe 39 :

- Gwenaël LASSALLE
- Théau NGUYEN-CALLOT
- Milyan VUKOMANOVIC

Tutrice :
Manon EVAIN

Sommaire

Introduction **2**

1 Présentation du projet **3**

1.1 Contexte 3

1.2 Besoins fonctionnels 3

1.3 Présentation des données 3

2 Conception de l'application **4**

2.1 Modélisation 4

2.2 Architecture générale 8

2.3 Choix technologiques 9

3 Réalisation **10**

3.1 Organisation du travail 10

3.2 Développement 10

3.3 Difficultés rencontrées 10

4 Tests et validation **11**

4.1 Stratégie de test 11

4.2 Quelques résultats 13

4.3 Limites de l'application 16

Conclusion **17**

Annexes **18**

Introduction

Le domaine du sport de haut niveau a connu, au cours de la dernière décennie, une révolution majeure. Qu'il s'agisse de l'optimisation des performances des athlètes, de l'analyse tactique ou de l'engagement des passionnés, la donnée est devenue un pilier central de l'écosystème sportif mondial. Cependant, la variété de disciplines allant des sports de ballon collectifs aux sports de raquette individuels pose un défi informatique de taille : comment concevoir un système capable d'unifier et de traiter des données aux structures hétérogènes ?

Dans le cadre de ce Projet de Traitement de Données, nous avons développé « **TGM Statistiques** », une plateforme logicielle conçue pour centraliser et analyser les statistiques de deux disciplines majeures : le basketball et le tennis. Se concentrer sur seulement deux sports est un choix fort : cela nous permet de privilégier la profondeur d'analyse à la quantité. En proposant des indicateurs très détaillés sur le déroulé précis des rencontres, à l'instar d'applications de référence comme *Flashscore*, notre outil offre une exploitation fine des données, utile tant pour les analystes techniques que pour les passionnés de sport. L'objectif de ce projet n'est pas seulement de fournir des indicateurs de performance, mais de proposer une architecture logicielle robuste et générique, fondée sur les principes de la programmation orientée objet.

Notre démarche s'articule autour de trois axes principaux présentés dans ce rapport. Dans un premier temps, nous décrivons les jeux de données utilisés et les méthodes d'ingestion flexibles que nous avons mises en place pour pallier la diversité des formats CSV. Ensuite, nous exposons la modélisation logicielle à travers nos diagrammes de classes UML, justifiant nos choix de conception pour garantir l'extensibilité du programme. Enfin, nous présentons les fonctionnalités de notre interface de consultation, permettant d'explorer les historiques de matchs et les classements de manière interactive.

1 Présentation du projet

1.1 Contexte

Le présent projet s'inscrit dans le cadre du Projet de traitement de données. Le cahier des charges initial nous imposait de concevoir une application Python respectant les principes fondamentaux de la programmation orientée objet, avec une contrainte de généralisabilité absolue. Le code devait être capable de traiter des jeux de données variés, permettant ainsi une grande adaptabilité. Le but de notre application est donc de proposer un système faisant abstraction des règles spécifiques à chaque sport pour offrir un outil d'analyse statistique universel. À travers ce projet, nous avons voulu démontrer qu'une modélisation logicielle rigoureuse permet de transformer des fichiers CSV diverses en une source d'information cohérente, permettant de consulter des classements et des historiques de performances sur quelques disciplines.

1.2 Besoins fonctionnels

L'application a pour mission principale de centraliser et de transformer des données sportives brutes en informations exploitables pour l'utilisateur. Pour cela, elle doit répondre en premier lieu à un besoin d'importation et de normalisation, capable de lire différents flux de données tout en nettoyant les éventuelles erreurs ou informations manquantes pour garantir une base de travail fiable. Une fois ces données intégrées, le système assure le calcul automatique de statistiques personnalisées, allant des bilans classiques de victoires et de défaites aux indicateurs de performance plus techniques et spécifiques à la discipline concernée. L'accès à ces informations est ensuite facilité par une interface de consultation intuitive qui permet à l'utilisateur de naviguer dans les données, d'effectuer des recherches ciblées sur des joueurs ou des équipes, de filtrer les résultats selon divers critères et de consulter des historiques complets.

1.3 Présentation des données

Nous avons choisi d'utiliser seulement certains jeux de données parmi ceux proposés afin de modéliser notre application. Notre sélection a été pensée pour diversifier au maximum notre approche : nous avons ainsi veillé à inclure à la fois un sport collectif et un sport individuel, tout en intégrant des données issues à la fois du sport masculin et du sport féminin. Dans cette optique, nous avons travaillé à partir des fichiers suivants :

- **Basket-ball** :
  - `game.csv` : Données sur les matchs NBA.
  - `player.csv` : Informations sur les joueurs NBA.
  - `team.csv` : Informations sur les équipes NBA.
- **Tennis** :
  - `atp_matches_2024.csv` : Résultats détaillés des matchs du circuit ATP.
  - `atp_players_2024.csv` : Informations sur les joueurs du circuit ATP.
  - `wta_matches_2024.csv` : Résultats détaillés des matchs du circuit WTA.
  - `wta_players_2024.csv` : Informations sur les joueuses du circuit WTA.

Le choix du basket-ball et du tennis s'appuie sur la richesse et la précision des données statistiques qu'ils génèrent, offrant une analyse bien plus fine que le simple score final. Par exemple, au basket-ball, il est possible d'évaluer les pourcentages de réussite aux tirs, que ce soit à l'échelle collective ou individuelle. De la même manière, le tennis offre des indicateurs techniques cruciaux tels que le nombre d'aces, de doubles fautes ou d'erreurs directes. L'exploitation de ces statistiques avancées permet de dépasser le résultat brut pour comprendre le déroulement réel du match. Cette approche est particulièrement pertinente pour les analystes souhaitant s'appuyer sur des données détaillées afin de comprendre les tendances de jeu et d'anticiper les performances lors de rencontres futures.

Avant de débuter l'analyse de nos différents jeux de données sportives, nous avons réalisé une phase d'exploration approfondie afin d'en garantir la fiabilité. Cette étape a consisté à identifier d'éventuels doublons et à recenser les valeurs manquantes qui auraient pu provoquer des erreurs lors de certaines demandes. Ces vérifications préliminaires nous permettent de confirmer que les bases de données sont exploitables.

2 Conception de l'application

2.1 Modélisation

Le projet est structuré autour de plusieurs classes regroupées dans le package `pkg`.

Tout d'abord la classe **Joueur** et ses spécialisations **JoueurBasket** et **JoueurTennis**, ainsi que la classe **Match** avec son dictionnaire stats générique, et la classe **Equipe** qui agrège un effectif de joueurs toutes représentées dans le diagramme ci-dessus.

La classe abstraite **BaseAdapter** impose le fonctionnement via les méthodes *adapt()* et *to_row()*, dont héritent **BasketJoueurAdapter** et **TennisJoueurAdapter** pour les joueurs, ainsi que leurs équivalents pour les matchs. Ce découpage permet d'absorber les différences de format entre les datasets NBA et Tennis sans impacter le reste du code.

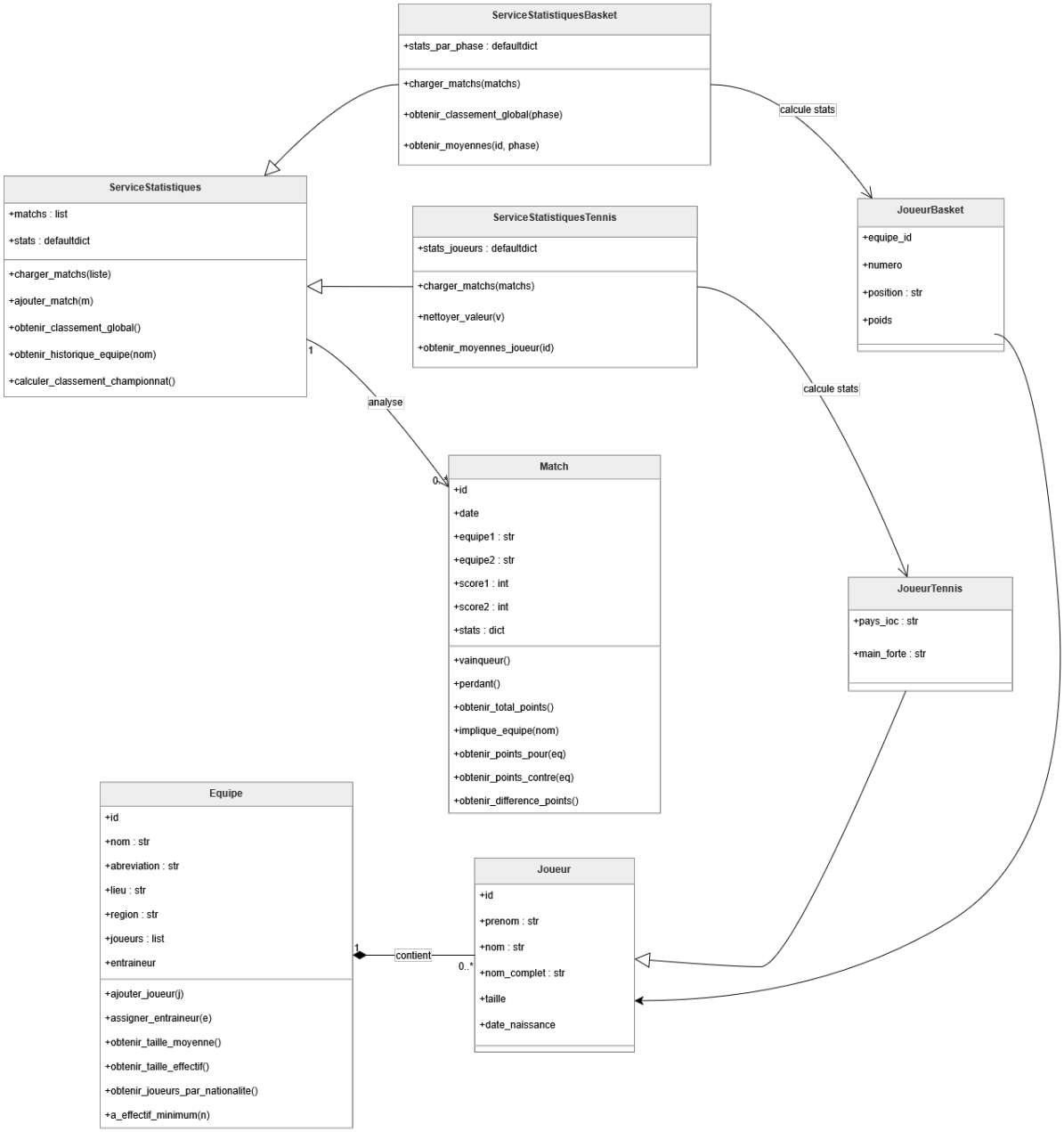

Figure 1: Diagramme de classe

DataRepository, qui orchestre le chargement et la sauvegarde des fichiers CSV en déléguant la conversion à l'adaptateur correspondant.

En finalement les services avec **ServiceAnnuaireJoueurs** qui gère les profils et expose des filtres par équipe ou par pays. **ServiceStatistiques**, **ServiceStatistiquesBasket** et **ServiceStatistiquesTennis** représentées dans le diagramme prennent en charge les classements et moyennes. Enfin, **ServiceApplication** orchestre la navigation de l'interface console en s'appuyant sur l'ensemble de ces services.

Le diagramme de cas d'utilisation de notre application fait apparaître un unique acteur **l'Utilisateur** qui interagit avec l'ensemble du système via un menu console.

Au lancement, l'utilisateur choisit un sport.

S'il sélectionne le module *NBA*, un sous-menu lui propose deux options. La première lui permet de consulter le classement et les statistiques avancées des équipes : il choisit d'abord une phase (Saison Régulière ou Playoffs), puis une conférence (Est, Ouest ou Global). L'application charge alors les matchs via **DataRepository**, calcule les victoires et les moyennes par équipe (points marqués et encaissés, rebonds, passes, interceptions, contres, pourcentages de tir) et affiche le classement filtré. La seconde option lui permet d'explorer les effectifs : il sélectionne une équipe parmi la liste organisée par conférence, consulte son effectif trié par numéro de maillot, puis choisit un joueur pour accéder à sa carte détaillée (position, taille, poids, date de naissance).

S'il sélectionne le module *Tennis*, un sous-menu lui propose de choisir entre le circuit ATP et le circuit WTA. Dans les deux cas, le déroulé est identique : l'application affiche les codes pays disponibles, ce qui nécessite de connaître le pays du joueur que l'on recherche, ce qui peut être contraignant. Toutefois c'est la méthode qui nous paraissait être la plus pertinente pour avoir accès à un maximum de joueurs. L'utilisateur saisit donc un pays, puis choisit un joueur parmi ceux représentant le pays choisi. Il accède alors à la fiche complète du joueur, comprenant ses statistiques agrégées sur la saison (victoires, défaites, aces par match, double-fautes, temps de jeu moyen, balles de break sauvées et converties) ainsi que son palmarès des tournois remportés en 2024.

Enfin Quitter l'application est accessible à tout moment depuis le menu principal

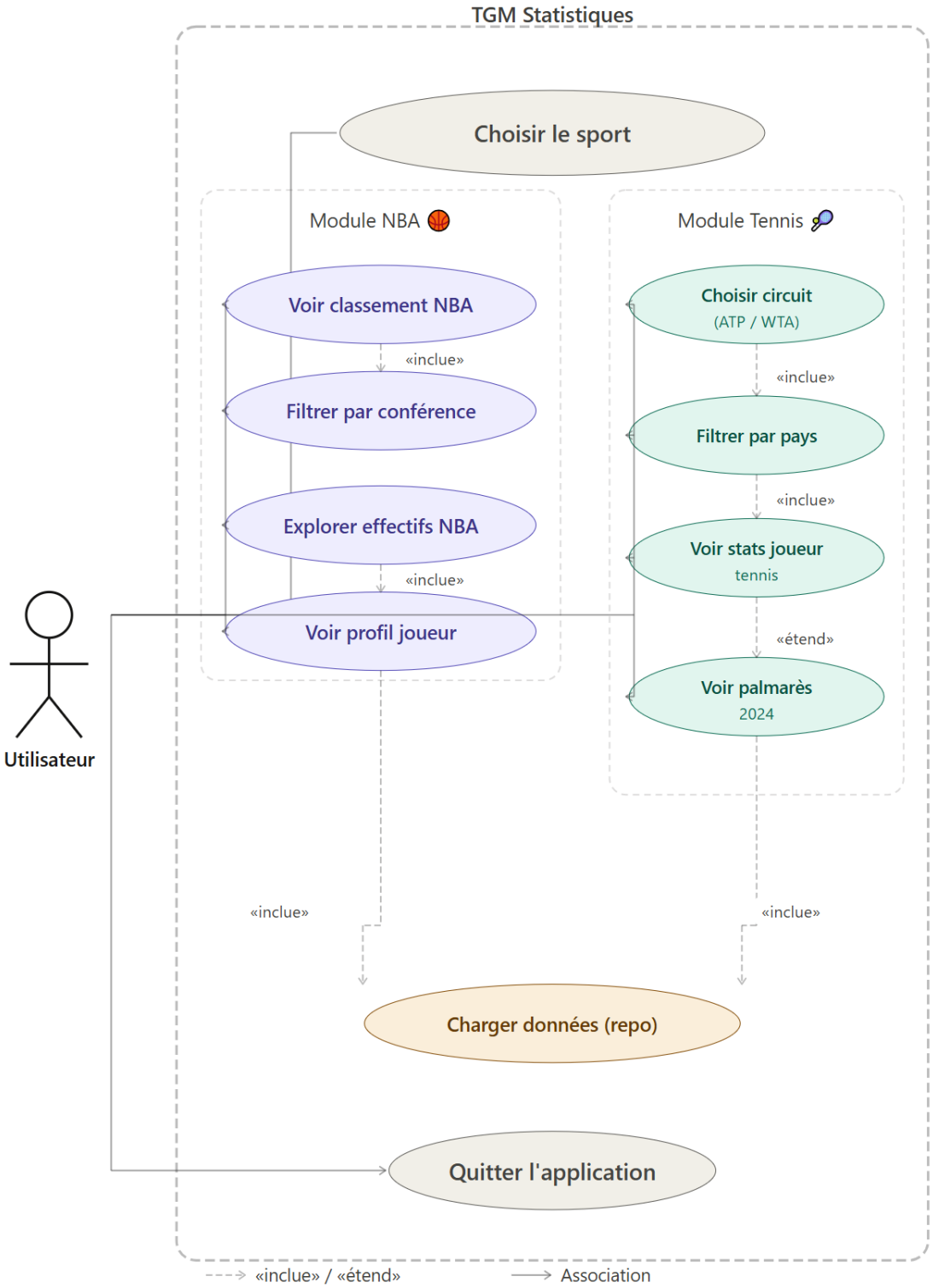

Figure 2: Diagramme de cas d'utilisation

2.2 Architecture générale

Notre projet repose sur une structure orientée objet qui privilégie la modularité et l'organisation. L'architecture repose sur plusieurs modules, chacun ayant une responsabilité bien définie :

```
projet/
  main.py
  pyproject.toml
  README.md
  pkg/
      __init__.py
      models/
          __init__.py
          joueur.py
          equipe.py
          match.py
      adapter/
          __init__.py
          base_adapter.py
          generic_joueur_adapter.py
          generic_equipe_adapter.py
          generic_match_adapter.py
      repository/
          __init__.py
          data_repository.py
      config/
          __init__.py
          dataset_configuration.py
      services/
          __init__.py
          service_application.py
          service_statistiques.py
          service_annuaire_joueur.py
          services_matchs.py
      tests/
          __init__.py
          test_models.py
          test_adapters.py
          test_services.py
          test_repository.py
```

L'architecture de code repose sur quatre piliers stratégiques qui justifient sa pertinence pour la modélisation :

Le Modèle de Données Typé : Plutôt que de manipuler des structures de données brutes (comme des dictionnaires ou des simples lignes de CSV), l'architecture utilise des objets métiers (**Joueur**, **Equipe**, **Match**). Ce choix permet de valider les données dès leur instanciation et d'utiliser l'héritage pour gérer les spécificités par sport (ex: **JoueurBasket** vs **JoueurTennis**). Cela sécurise la modélisation en garantissant que chaque entité possède les attributs nécessaires avant d'atteindre la logique de calcul.

Le Pattern Adapter : c'est le cœur de la flexibilité du projet. La couche **adapter/** agit comme une passerelle de traduction entre le format brut des fichiers CSV et les objets Python. En utilisant une classe abstraite **BaseAdapter**, l'architecture permet d'ajouter de nouveaux datasets sans jamais modifier le code existant. Cette abstraction sépare la structure technique des données (comment elles sont stockées) de leur sens métier (ce qu'elles représentent).

Le Découplage par le Repository et les Services :

le **DataRepository** centralise l'accès aux fichiers via Pandas, isolant ainsi la complexité technique de la lecture/écriture.

Les **Services** (statistiques, annuaire, matchs) contiennent la logique métier pure. Ils sont totalement indépendants de la source de données : ils reçoivent des objets déjà formattés et se concentrent uniquement sur les calculs et les fonctionnalités utilisateurs. Ce découplage permet de modifier l'algorithme d'un classement sans risquer de casser la lecture d'un fichier.

La Configurabilité et Maintenabilité : La présence d'un module **config/** dédié montre une volonté d'industrialisation du code. En centralisant les chemins de fichiers et les séparateurs, on permet au programme d'être multi-sport de manière dynamique. Enfin, la structure incluant un répertoire **tests/** et un fichier **pyproject.toml** assure que le projet suit les standards modernes de développement (tests unitaires, gestion propre des dépendances et typage avec Mypy), garantissant la fiabilité des résultats produits par le modèle.

2.3 Choix technologiques

Le choix de Python 3.12 associé à une architecture Repository/Adapter/Service et à la Programmation Orientée Objet permet de construire une application modulaire, où les classes abstraites facilitent l'intégration de nouvelles sources de données sportives sans fragiliser le système. La bibliothèque *Pandas* est ici indispensable pour manipuler les fichiers CSV : elle transforme les données brutes en DataFrames pour permettre un filtrage, un tri et une organisation ultra-rapide des performances. Pour la partie analytique, le module math assure la précision des calculs techniques complexes tandis que l'outil *defaultdict* du module collections simplifie la création de dictionnaires de statistiques en gérant automatiquement l'initialisation des scores pour chaque nouveau joueur ou équipe. Enfin, la fiabilité du code est garantie par *pytest* et *unittest.mock* : les MagicMock agissent comme des "doublures"

qui imitent des fichiers, tandis que les patches détournent les appels du programme vers ces faux composants plutôt que vers le disque dur. Concrètement, cela permet de faire croire au programme qu'il lit un fichier réel pour vérifier instantanément s'il calcule bien les classements, même dans des cas compliqués ou face à des erreurs (comme un fichier corrompu), sans avoir besoin de créer ou de manipuler de vrais documents sur l'ordinateur.

3 Réalisation

3.1 Organisation du travail

L'organisation de ce projet a débuté par une phase de réflexion collective afin de structurer les axes majeurs de développement de l'application. Cette première étape de modélisation nous a permis d'accorder nos visions techniques avant que l'arrivée concrète des jeux de données ne nous permette d'affiner la structure des classes et la logique des méthodes de traitement.

Une fois ce cadre méthodologique établi, nous avons réparti le développement technique de la façon suivante : Théau s'est occupé de la gestion du flux de données, développant les scripts d'insertion et les adaptateurs nécessaires à l'intégration, tandis que Milyan s'est concentré sur l'architecture logicielle en codant l'ensemble des classes structurantes du système. En parallèle, Gwenaël a pris en charge le contrôle qualité à travers la vérification de la propreté des données, tout en concevant l'interface utilisateur afin de permettre une accessibilité simple aux données.

Parallèlement au développement, nous avons mis en place une stratégie de tests modulaires pour garantir la fiabilité de chaque couche de notre application. Théau a pris en charge la validation des flux d'entrée via les tests d'adaptateurs (*test_adapters.py*), Milyan a assuré la conformité des entités métier à travers les tests de modèles (*test_models.py*), tandis que Gwenaël s'est occupé de valider la logique globale et les calculs statistiques dans les tests de services (*test_services.py*).

Afin de fluidifier notre collaboration et d'assurer la cohérence de notre développement, nous avons utilisé Git. Cet outil a été indispensable pour travailler simultanément sur le même code source, gérer les fusions de nos travaux respectifs et conserver un historique rigoureux de chaque étape du projet.

Nous avons ensuite rédigé le rapport tous les trois, en choisissant d'utiliser l'outil Quarto.

3.2 Développement

3.3 Difficultés rencontrées

L'une des principales difficultés de ce projet a résidé dans son démarrage : en l'absence de sujet imposé et n'ayant pas encore les jeux de données à notre disposition, il a été particulièrement

complexe de lancer une dynamique de travail concrète. Cette phase d'incertitude a rendu difficile la définition du concept même de notre future application, car il est délicat de structurer une architecture logicielle ou d'imaginer des fonctionnalités pertinentes sans connaître la nature exacte, le volume et les contraintes des données à traiter.

La gestion de l'héritage et des classes abstraites a représenté un défi structurel majeur, notamment pour concevoir une hiérarchie cohérente entre les joueurs et les adaptateurs spécifiques au basketball et au tennis. Pour résoudre cette difficulté, nous avons mené une réflexion approfondie afin de bien identifier les attributs communs et les besoins spécifiques à chaque sport, ce qui a permis d'implémenter une structure solide évitant toute duplication de code.

Le traitement des données « sales » dans les fichiers CSV réels, comme les valeurs manquantes ou les formats incohérents, a nécessité une attention particulière. Nous avons résolu ce problème en intégrant des mécanismes de nettoyage et de conversion directement dans les classes *StatistiqueJoueur* et *ServiceAnnuaireJoueurs*, assurant la transformation systématique des données (conversion en float ou passage des lbs aux kg) et le filtrage des valeurs inexploitables.

La séparation des responsabilités entre les différents services (Statistiques, Annuaire, Matchs) est devenue de plus en plus délicate à mesure que la taille du projet augmentait. Pour éviter que leurs fonctions ne se chevauchent, nous avons mené un exercice d'architecture logicielle rigoureux, redéfinissant précisément le rôle de chaque service pour garantir que chaque composant ne s'occupe que de sa mission spécifique.

Enfin, le couplage entre les adaptateurs et les modèles de données présentait une fragilité : le moindre changement dans un nom de colonne CSV pouvait bloquer l'application. Nous avons stabilisé ce point en utilisant des valeurs par défaut lors de la récupération des données (via la méthode row.get(col, "N/A")), apportant ainsi la rigueur nécessaire pour que le programme reste fonctionnel et robuste même en cas de données manquantes ou de modifications imprévues des fichiers sources.

4 Tests et validation

4.1 Stratégie de test

Pour garantir la fiabilité de notre application, nous avons mis en place une stratégie de tests unitaires et d'intégration basée sur le framework Pytest. L'objectif est double : assurer l'exactitude des calculs statistiques et garantir la robustesse des processus d'importation.

La validation commence avec les modèles et les adaptateurs. Les tests sur les modèles garantissent que chaque objet métier, comme un match ou un joueur, est instancié avec des données saines et que ses propriétés calculées (par exemple, la concaténation du nom complet) se comportent de manière prévisible. Parallèlement, la couche des adaptateurs fait l'objet

d'une attention critique. Nous y testons le mapping entre le format brut des dictionnaires issus du CSV et nos objets Python, en mettant à l'épreuve la gestion des valeurs manquantes et la précision des conversions techniques, telles que la transformation des poids de lbs en kg.

Le cœur du programme, regroupé dans la partie services, est quant à lui vérifié par des tests précis sur les calculs et les règles de gestion. Ces tests permettent de contrôler que les classements des équipes, les moyennes de points ou encore l'historique des joueurs sont calculés sans aucune erreur. Pour que ces vérifications soient fiables et rapides, nous utilisons des simulations (appelées "Mocks") qui remplacent les fichiers réels par des données fictives mais contrôlées. Cela nous permet de tester le "cerveau" de l'application de façon isolée, en étant certains que si un résultat est faux, cela vient d'une erreur de calcul et non d'un problème technique lié au chargement des données.

Enfin, la fiabilité de la persistance et de l'accès aux données est assurée par des tests aux frontières au niveau des repositories et des importeurs. Nous utilisons des environnements de fichiers temporaires via *tmp_path*. Cette méthode nous permet de vérifier, dans des conditions quasi réelles, que le système gère correctement les différents séparateurs et l'encodage des fichiers CSV. Cette stratégie globale assure ainsi une interface fluide et sécurisée entre le stockage physique et la logique métier de l'application.

4.2 Quelques résultats

L'expérience utilisateur débute par une interface d'accueil intuitive permettant de sélectionner le sport à analyser, orientant ainsi immédiatement le flux de données vers le domaine de compétences souhaité. Ce point d'entrée unique assure une navigation fluide vers les fonctionnalités de statistiques et de recherche spécifiques au basketball ou au tennis.

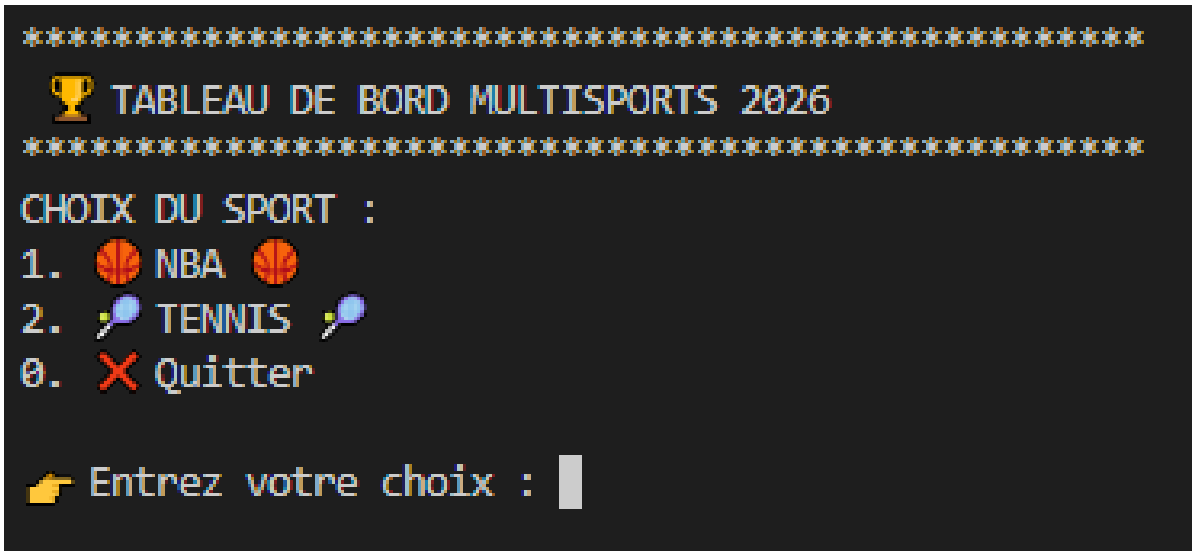

Figure 3: Menu permettant le choix du sport

Prenons l'exemple du tennis, l'utilisateur a maintenant le choix entre le circuit ATP et WTA

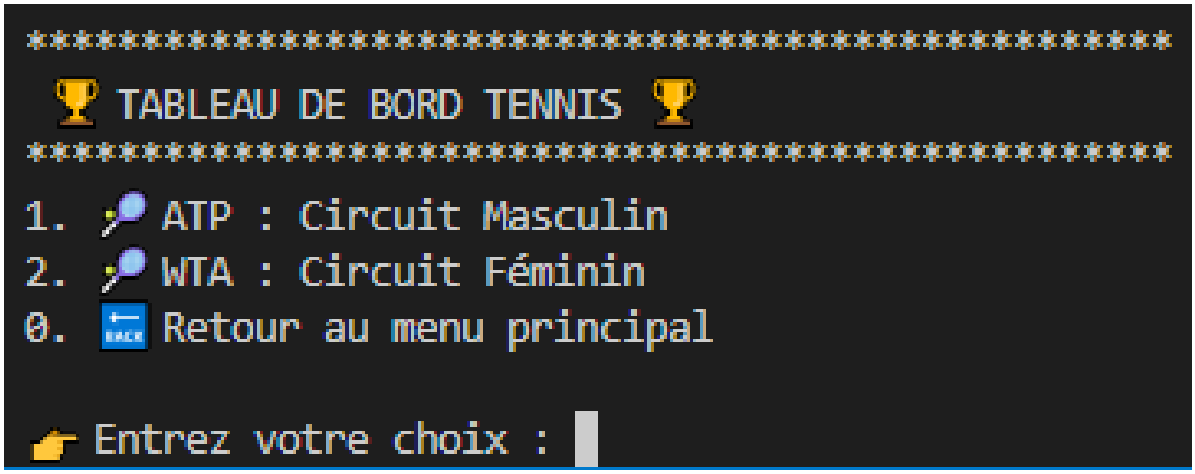

Figure 4: Menu permettant le choix du circuit

L'utilisateur a ensuite le choix entre les pays représentés sur le circuit, suite à cela, la liste des

joueurs de nationalité du pays choisi apparait. Choisissons l'Italie comme pays puis le joueur Jannik Sinner.

```
★★★
```

```
Name                                          Stmts   Miss  Cover
-----------------------------------------------------------------
pkg\adapter\base_adapter.py                       8      2    75%
pkg\adapter\generic_equipe_adapter.py            14      0   100%
pkg\adapter\generic_joueur_adapter.py            27      1    96%
pkg\adapter\generic_match_adapter.py             41      1    98%
pkg\config\dataset_configuration.py              15      0   100%
pkg\models\equipe.py                             29      2    93%
pkg\models\joueur.py                             20      0   100%
pkg\models\match.py                              43      0   100%
pkg\repository\data_repository.py                15      0   100%
pkg\services\service_annuaire_joueur.py          40      0   100%
pkg\services\service_statistiques.py            168      4    98%
pkg\services\services_matchs.py                  14      4    71%
-----------------------------------------------------------------
TOTAL                                           434     14    97%
133 passed in 2.22s
```

Figure 6: Couverture des tests

Afin de garantir la fiabilité et la robustesse de l'application, une attention particulière a été portée à la couverture de tests unitaires et d'intégration. En cohérence avec une approche de développement dirigé par les tests, l'intégralité des modules constituant l'architecture du projet a été soumise à une vérification rigoureuse. Les résultats obtenus, illustrés par le rapport de couverture, affichent un score global de 97 %. Ce niveau de couverture élevé témoigne d'une validation quasi exhaustive des différents chemins d'exécution du code et des cas limites. Le succès de l'ensemble des tests exécutés confirme la stabilité des services implémentés, assurant ainsi une base solide pour la maintenance et les évolutions futures du logiciel.

(faire résultats des linters)

4.3 Limites de l'application

Le choix de nous concentrer exclusivement sur deux sports rend l'application particulièrement spécifique. L'interface utilisateur ayant été finement adaptée aux caractéristiques propres de ces disciplines, cette spécialisation crée une contrainte majeure en termes d'évolutivité : l'ajout d'un nouveau sport s'avère complexe. En effet, chaque nouvelle catégorie nécessiterait une adaptation de l'interface pour répondre à ses spécificités, ce qui limite sa capacité à s'ouvrir à d'autres domaines sportifs sans un travail de développement supplémentaire conséquent.

Par ailleurs, l'ergonomie de l'application constitue une limite importante. Le choix d'une interface en ligne de commande (CLI), bien que parfaitement fonctionnelle et efficace pour le traitement des données, manque d'attrait visuel et de convivialité. L'absence d'interface graphique (GUI) peut rendre la navigation moins intuitive pour un utilisateur non habitué au terminal. Bien que cet environnement épuré permette de se concentrer sur la logique métier, une évolution vers une solution plus moderne et esthétique améliorerait significativement l'expérience utilisateur.

Enfin, le périmètre fonctionnel actuel restreint les possibilités d'analyse globale. Certaines statistiques avancées manquent encore à l'appel et les critères de recherche demeurent limités. Pour le tennis, par exemple, l'application permet de consulter les résultats uniquement par joueur, sans offrir la possibilité d'effectuer une recherche par tournoi ou par type de surface. L'ajout de filtres multicritères et de statistiques technico-tactiques plus poussées (comme le taux de réussite au premier service ou les fautes directes) constituerait une amélioration majeure pour transformer cet outil de consultation en un véritable instrument d'analyse sportive.

Conclusion

Ce projet nous a permis de concevoir et de développer « **TGM Statistiques** », une plateforme d'analyse statistique sportive couvrant deux disciplines complémentaires : le basketball (sport collectif) et le tennis (individuel). En combinant une modélisation orientée objet rigoureuse et une architecture modulaire fondée sur les patterns Adapter, Repository et Service, nous avons réussi à transformer des fichiers CSV aux formats hétérogènes en une source d'information cohérente et exploitable par l'utilisateur.

Sur le plan technique, la mise en place d'une hiérarchie de classes abstraites a constitué le cœur de notre démarche de conception. Ce choix architectural a permis de garantir l'extensibilité du système tout en maintenant une séparation claire des responsabilités entre les différentes couches de l'application. La stratégie de tests unitaires adoptée, avec 133 tests validés pour une couverture globale de 97 %, témoigne de la robustesse du code produit et de notre volonté de livrer une application fiable et maintenable.

Sur le plan analytique, l'exploitation approfondie des données NBA et tennis nous a permis de proposer des indicateurs de performance dépassant le simple résultat brut : classements par phase et par conférence, moyennes avancées par équipe, statistiques individuelles détaillées et palmarès des joueurs. Cette profondeur d'analyse, inspirée d'outils de référence comme Flashscore, représente selon nous la principale valeur ajoutée de notre application.

Ce projet nous a également permis de consolider des compétences transversales essentielles, allant de la gestion de versions avec *Git* à la rédaction collaborative sous *Quarto*, en passant par la conception *UML* et le développement piloté par les tests. Les difficultés rencontrées, notamment la gestion des données manquantes et la séparation des responsabilités entre services, ont été autant d'occasions d'approfondir notre réflexion architecturale et de produire un code plus robuste.

Enfin, si l'application présente aujourd'hui des limites en termes d'évolutivité vers de nouveaux sports, les fondations posées constituent une base solide et réutilisable sur laquelle il serait possible de s'appuyer pour étendre le système à d'autres disciplines, en implémentant de nouveaux adaptateurs et services sans remettre en cause l'architecture existante.

Annexes

Voici quelques annexes sur les résultats du basket.

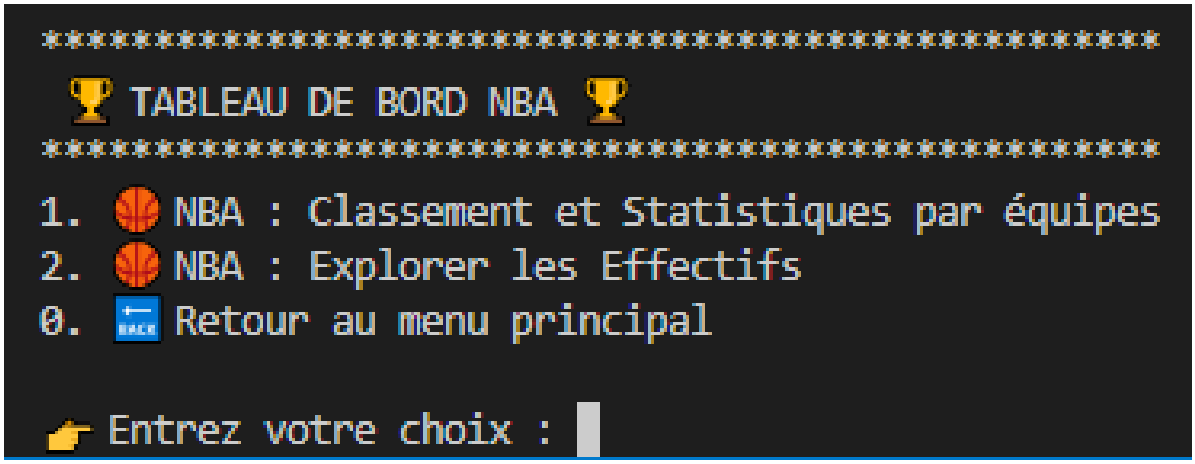

Figure 7: Menu NBA

--- RÉSULTATS POUR : REGULAR SEASON | GLOBALE ---

#	Équipe	Vic	Pts	Enc	Reb	Ast	Stl	Blk	2P%	3P%	LF%
1	Milwaukee Bucks	58	116.9	113.3	48.6	25.8	6.4	4.9	55.7	36.8	74.3
2	Boston Celtics	57	117.9	111.4	45.3	26.7	6.4	5.2	56.7	37.7	81.2
3	Philadelphia 76ers	54	115.2	110.9	40.9	25.2	7.7	4.7	55.1	38.7	83.5
4	Denver Nuggets	53	115.8	112.5	43.0	28.9	7.5	4.5	57.5	37.9	75.1
5	Cleveland Cavaliers	51	112.3	106.9	41.1	24.9	7.1	4.7	55.9	36.7	78.0
6	Memphis Grizzlies	51	116.9	113.0	46.6	26.0	8.3	5.8	54.8	35.1	73.3
7	Sacramento Kings	48	120.7	118.1	42.5	27.3	7.0	3.4	58.6	36.9	79.0
8	New York Knicks	47	116.0	113.1	46.6	22.9	6.4	4.1	54.7	35.4	76.1
9	Brooklyn Nets	45	113.4	112.5	40.5	25.5	7.1	6.2	55.9	37.8	80.0
10	Phoenix Suns	45	113.6	111.6	44.2	27.3	7.1	5.3	52.0	37.4	79.3
11	Golden State Warriors	44	118.9	117.1	44.6	29.8	7.2	3.9	56.4	38.5	79.4
12	Miami Heat	44	109.5	109.8	40.6	23.8	8.0	3.0	54.0	34.4	83.1
13	Los Angeles Clippers	44	113.6	113.1	43.2	23.9	7.1	4.4	53.9	38.1	78.1
14	Los Angeles Lakers	43	117.2	116.6	45.7	25.3	6.4	4.6	55.5	34.6	77.5
15	New Orleans Pelicans	42	114.4	112.5	43.7	25.9	8.3	4.1	54.1	36.4	79.2
16	Minnesota Timberwolves	42	115.8	115.8	41.9	26.2	8.0	5.4	56.8	36.5	75.5
17	Toronto Raptors	41	112.9	111.4	43.0	23.9	9.4	5.2	52.5	33.5	78.4
18	Atlanta Hawks	41	118.4	118.1	44.4	25.0	7.1	4.9	54.8	35.2	81.8
19	Oklahoma City Thunder	40	117.5	116.4	43.6	24.4	8.2	4.2	53.0	35.6	80.9
20	Chicago Bulls	40	113.1	111.8	42.4	24.5	7.9	4.5	55.5	36.1	80.9
21	Dallas Mavericks	38	114.2	114.1	38.8	22.9	6.3	3.7	57.4	37.1	75.5
22	Utah Jazz	37	117.1	118.0	45.9	26.0	6.1	5.2	56.0	35.3	78.6
23	Indiana Pacers	35	116.3	119.5	41.5	27.0	7.7	5.8	54.0	36.7	79.0
24	Washington Wizards	35	113.2	114.4	43.6	25.4	6.8	5.2	55.9	35.6	78.5
25	Orlando Magic	34	111.4	114.0	43.2	23.2	7.4	4.7	53.9	34.6	78.4
26	Portland Trail Blazers	33	113.4	117.4	40.5	24.2	6.7	4.6	55.0	36.5	79.6
27	Charlotte Hornets	27	111.0	117.2	44.5	25.1	7.7	5.2	52.8	33.0	74.9
28	San Antonio Spurs	22	113.0	123.1	43.7	27.2	7.0	3.9	52.9	34.5	74.3
29	Houston Rockets	22	110.7	118.6	46.3	22.4	7.3	4.6	53.0	32.7	75.4
30	Detroit Pistons	17	110.3	118.5	42.4	23.0	7.0	3.8	51.6	35.1	77.1

Figure 8: Classement NBA global

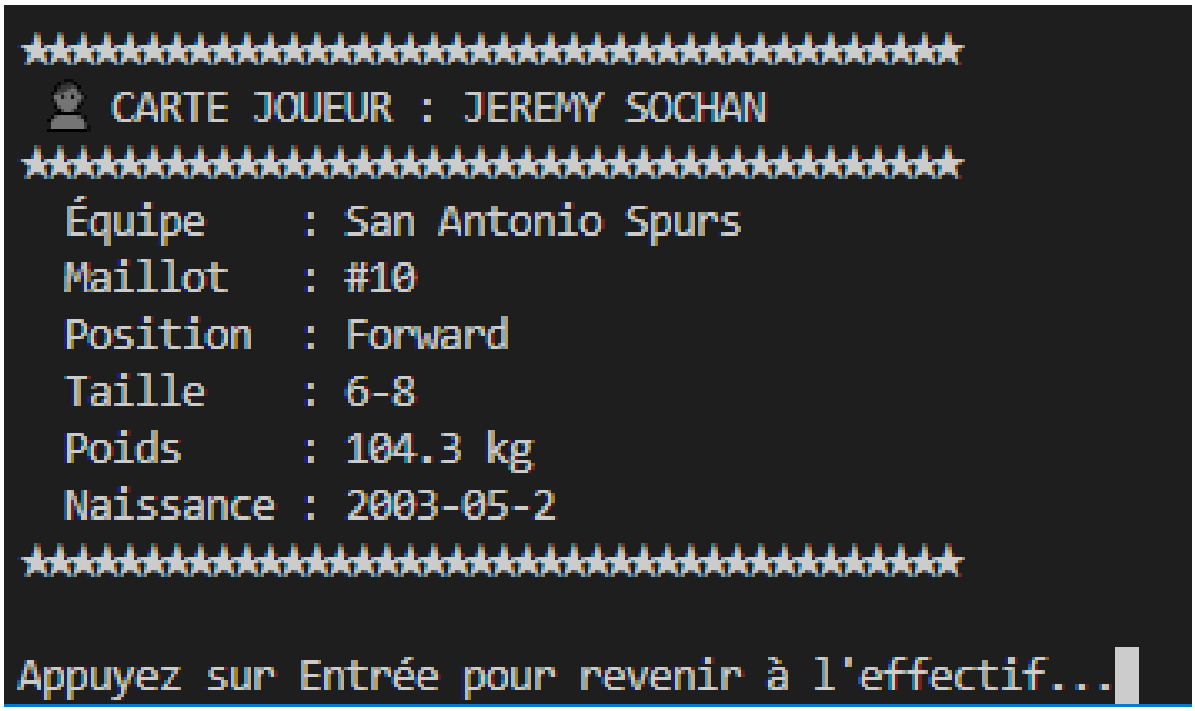

Figure 9: Carte de Jeremy Sochan